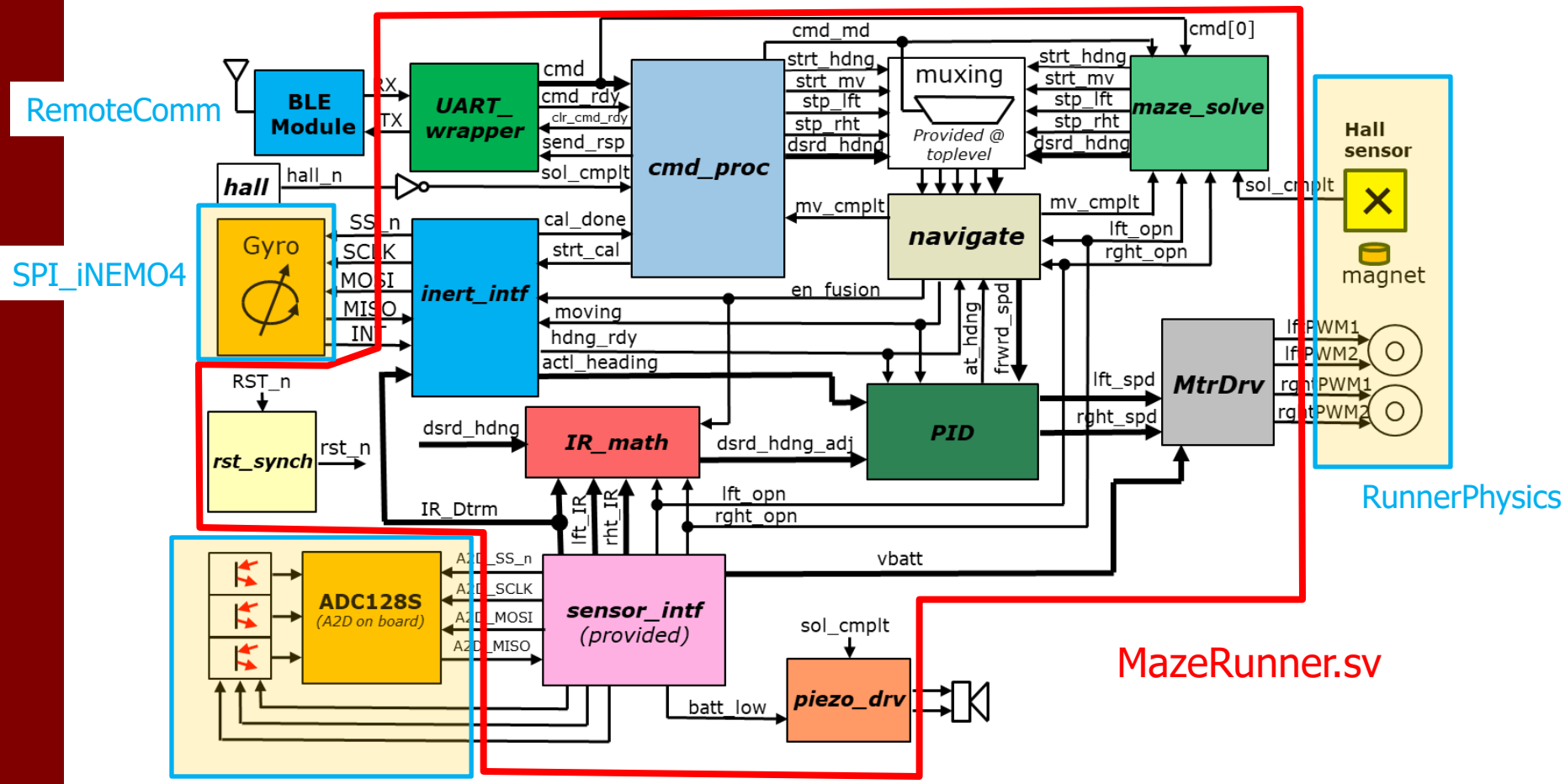
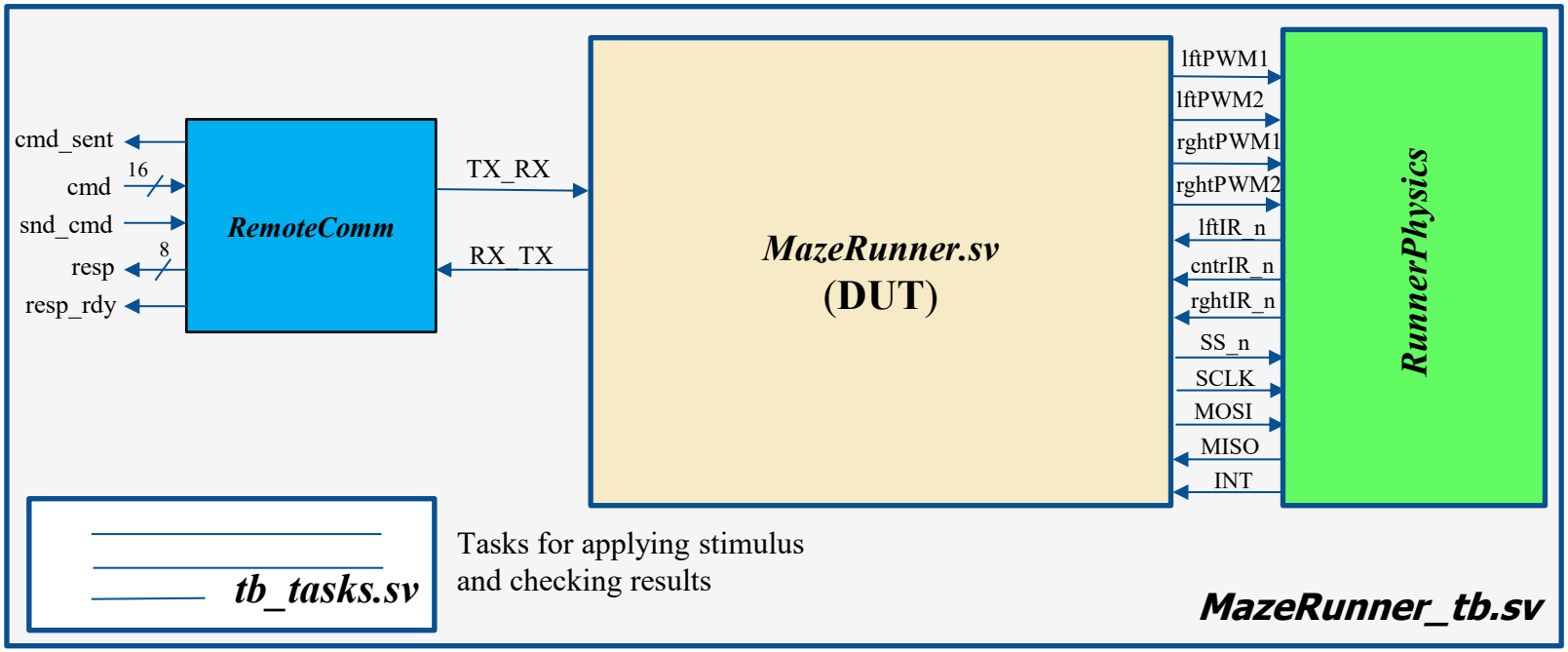
Project – MazeRunner

- We've completed all components of the MazeRunner!
- Need to put everything together & test full system



Project – MazeRunner Testing

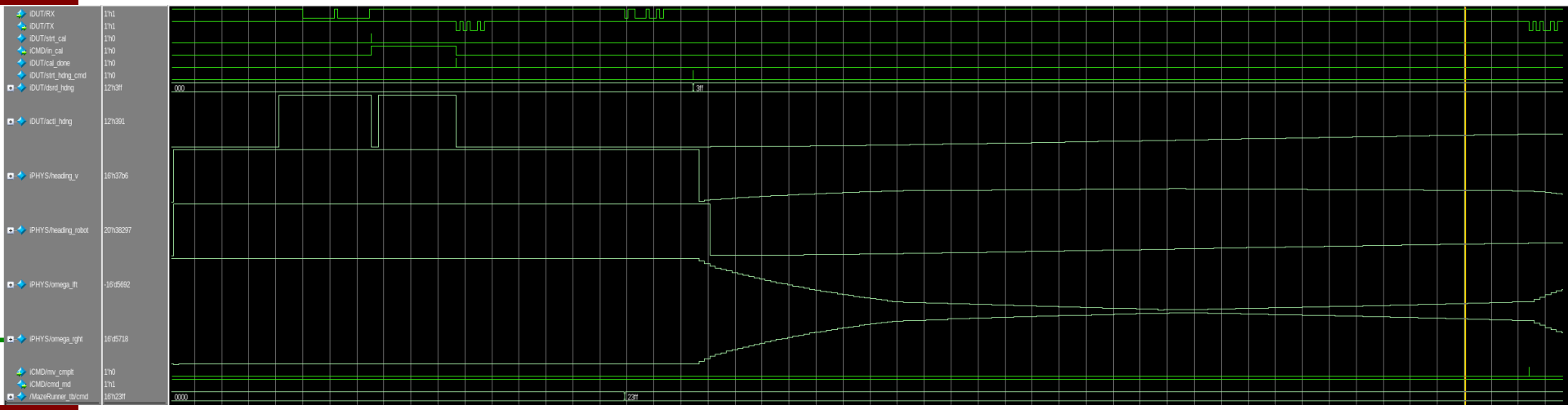
- We provide several models of the physical system so you can simulate & debug without the hardware
  - RunnerPhysics models the physics of MazeRunner and the maze
  - It monitors the PWM signals does a crude modeling of the physics, deriving quantities such as wheel velocities (**omega_lft/omega_right**) and yaw rate of the robot platform (**heading_v**), which in turn feeds the embedded iNEMO sensor model



Project – MazeRunner Testing

Example: calibrate, then change heading to west

- Calibrate
  - See strt_cal asserted, then in_cal stays high, then cal_done asserted, then pos Ack
- Change heading
  - See strt_hdng asserted
  - Wheel velocities change: omega_lft decreases (goes -ve), omega_right increases
  - Yaw rate of the robot platform (heading_v) should increase, then decrease
  - Heading of the robot (heading_robot or use actl_hdng in your DUT) should converge to the dsrd_hdng



Project – MazeRunner Testing

- Use tasks to make your testbench more readable & less repetitive
  - Initialize, SendCmd, ChkPosAck, … etc.

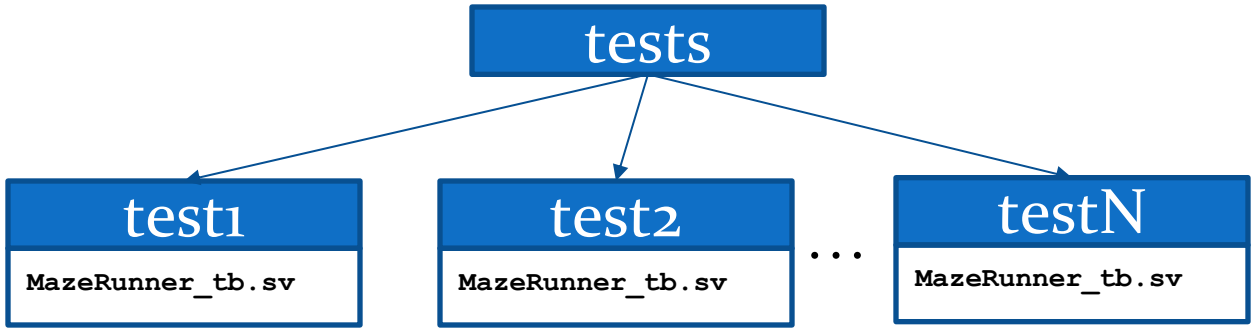
What to test? (start simple and work up to more complex)

- Do things initialize properly?
  - Are PWMs running at midrail values just after reset?
  - Does NEMO_setup eventually assert? (look inside RunnerPhysics)
- Send command to calibrate
  - Does cal_done assert?
  - Do you get a pos Ack?
- Send command to change heading to west
  - Does **error** signal change, does right wheel duty cycle increase relative to left?
  - Does heading start to change and eventually converge to desired heading?
- Send command to move forward
  - Does omega_sum (inside RunnerPhysics) ramp up?

- Ensure tests are "abbreviated" by having FAST_SIM=1
- vsim on Linux runs much faster than free versions on Windows

Project – MazeRunner Testing

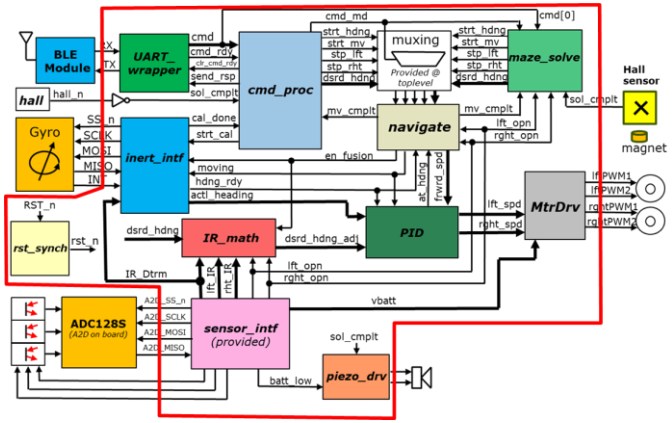
- Seems like a lot to test… How to attack it?
- Don't try to put everything in 1 giant test!
- Have a test suite
- Perhaps a directory of tests
  - Different versions of MazeRunner_tb.sv for each test



- Goal is to ensure the system works when we put it onto actual hardware
  - Think about what are the different scenarios you could have

Project – MazeRunner Synthesis

- You will synthesize MazeRunner.sv using the LVT library
- Target clock frequency is 363MHz (2.75ns period)
  - You will probably need to pipeline the design to meet timing



- Goal is to minimize area while still meeting timing

- You also need to demonstrate at least 1 of your tests running on the post-synthesis netlist successfully (MazeRunner.vg)